Day 64: The LSTM Forward Pass From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 2 | Complete study guide

Watch: CS231n Winter 2016: Lecture 10: Recurrent Neural Networks, Image Captioning, LSTM (Andrej Karpathy) | Implementing LSTM in numpy from scratch! (Ather Cheema)

Script: `day64_lstm_forward_pass.py` · every number in this guide is copied by program from `day64_run_output.txt`, the real output of that script.

Yesterday's LSTM cell did one step. Today it runs over whole sequences: a loop that carries **two** states from step to step and stores everything the backward pass will need tomorrow. The loop is checked against steps done by hand; padding is handled for both states; a faster version is timed; the forward pass is run for 200 steps to see what the cell state does; and the memory cost is measured. One result needed a second look: a speed-up that first measured about 8× turned out to be mostly this computer's matrix library, not the trick being tested. Nothing is trained today.

0. Where today fits

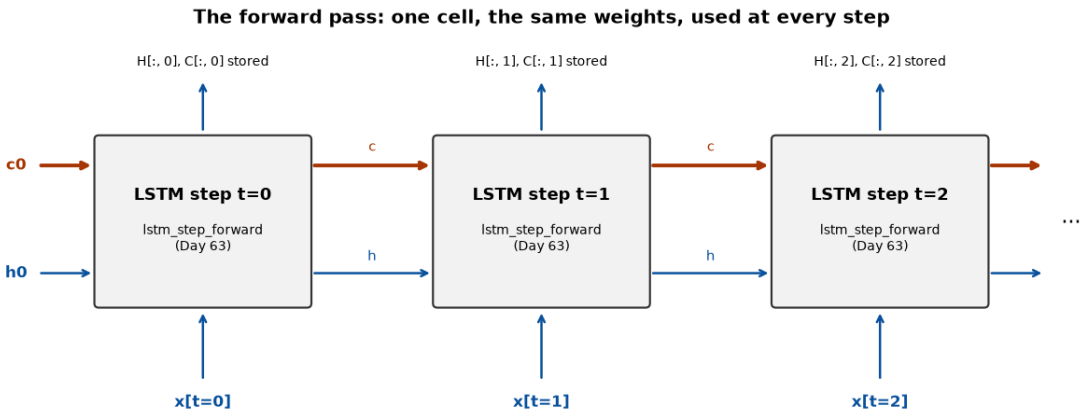
DEFINITION — start here, no prior knowledge assumed

A **forward pass** runs inputs through a network to get its outputs. For a recurrent network it means calling the same cell once per timestep, feeding each step the states the previous step produced. The **cache** is everything saved along the way so the backward pass can compute gradients without running the forward pass again.

REAL-WORLD SCENARIO

A relay race where each runner carries two batons: a heavy one that is passed on almost untouched (the cell state) and a light one each runner rewrites (the hidden state). A video camera records every hand-over (the cache), so that afterwards the coach can replay the race backwards and say exactly where time was lost.

SYSTEM DIAGRAM



The forward pass unrolled: the same Day 63 step at every position, with c (orange) and h (blue) handed from each step to the next. Every step's states and gates are stored.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The forward pass, checked against steps done by hand

DEFINITION — start here, no prior knowledge assumed

lstm_forward takes a batch of sequences, shape (examples, steps, inputs), and starting states h0 and c0. At each step it calls Day 63's `lstm_step_forward` and passes the new h **and** c on to the next step. It returns every hidden state, every cell state and the cache.

Because the same weights are used at every step, one set of weights handles a sequence of any length.

REAL-WORLD SCENARIO

A recipe followed once per guest at a dinner party: same instructions every time, but each time you also carry over what is left in the pot (the cell state) and what you last tasted (the hidden state). Whether there are 5 guests or 500, the recipe card does not get longer.

MATHEMATICS

$$\text{for } t = 0, 1, \ldots, T-1: \quad (h_t, c_t) = \text{step}(x_t, h_{t-1}, c_{t-1}; W, b) \quad \text{with } h_{-1} = h0,\ c_{-1} = c0$$

$$\text{cache} = \{ X, h0, c0, W, b, \text{mask}, \text{all } h_t, \text{all } c_t, \text{all } f_t, i_t, g_t, o_t \}$$

SPECIFICATION

Check	4 sequences, 3 steps, 2 inputs, 5 hidden units, random weights, non-zero h0 and c0. The loop is compared with three calls to `lstm_step_forward` made by hand.
Pass criterion	Identical: the loop does nothing but call the same function in the same order.
Lengths	One set of weights (3 inputs, 8 hidden units) run on sequences of 5, 50 and 500 steps.

Why this test

The loop adds very little code, but it is where states get mixed up: passing the old c instead of the new one, or storing h before it is updated. Comparing against hand-chained calls catches every such slip, and it is the same check Day 57 used for the RNN.

IMPLEMENTATION

```
def lstm_forward(X, h0, c0, W, b, mask=None):
    m, T, D = X.shape
    H = h0.shape[1]
```

```
    if mask is None:
        mask = np.ones((m, T))
    Hs, Cs = np.zeros((m, T, H)), np.zeros((m, T, H))
    F, I, G, O = (np.zeros((m, T, H)) for _ in range(4))
    h, c = h0, c0
    for t in range(T):
        h_new, c_new, (f, i, g, o) = lstm_step_forward(X[:, t, :], h, c, W, b)
        m_t = mask[:, t:t + 1]
        h = m_t * h_new + (1.0 - m_t) * h
        c = m_t * c_new + (1.0 - m_t) * c
        Hs[:, t], Cs[:, t] = h, c
        F[:, t], I[:, t], G[:, t], O[:, t] = f, i, g, o
    cache = dict(X=X, h0=h0, c0=c0, W=W, b=b, mask=mask, H=Hs, C=Cs, F=F, I=I, G=G, O=O)
    return Hs, Cs, cache
```

The subtle part: `F, I, G, O = (np.zeros(...) for _ in range(4))` makes four separate arrays. Writing `F = I = G = O = np.zeros(...)` would make four names for one array, and every gate would overwrite the others.

ACTUAL OUTPUT

```
 step   max |h loop - h by hand|   max |c loop - c by hand|
    0                  0.00e+00                  0.00e+00
    1                  0.00e+00                  0.00e+00
    2                  0.00e+00                  0.00e+00
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
what the cache holds for the backward pass (m=4, T=3, D=2, H=5):
  X     shape (4, 3, 2)
  h0    shape (4, 5)
  c0    shape (4, 5)
  W     shape (7, 20)
  b     shape (20,)
  mask  shape (4, 3)
  H     shape (4, 3, 5)
  C     shape (4, 3, 5)
  F     shape (4, 3, 5)
  I     shape (4, 3, 5)
  G     shape (4, 3, 5)
  O     shape (4, 3, 5)
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
any length works -- the same weights (384 numbers) on three different lengths:
  T=5    hidden states (2, 5, 8)     cell states (2, 5, 8)     largest |h| 0.285  largest |c|
      0.483
  T=50   hidden states (2, 50, 8)    cell states (2, 50, 8)    largest |h| 0.466  largest |c|
      0.794
  T=500  hidden states (2, 500, 8)   cell states (2, 500, 8)   largest |h| 0.464  largest |c|
      0.963
Actual output from this lesson's run.
```

What it means

The loop and the hand-chained steps agree exactly (0.00e+00) at all three steps. The cache holds twelve entries; six of them are full (examples × steps × hidden) arrays. The same 384 numbers run sequences of 5, 50 and 500 steps without any change. Over 500 steps the largest cell-state value is 0.963: with these random weights and the default forget bias, nothing grows out of control. Section 4 shows when it does.

WHY CHOOSE THIS?

The loop is a thin wrapper around a function already checked on Day 63, so almost everything new today can be trusted by construction.

WHY NOT THIS? (TRADEOFFS)

It stores every gate at every step, which is what makes it several times heavier in memory than an RNN (Section 5), and it cannot be parallelised across time: step t needs step t–1.

2. Padding with two states

DEFINITION — start here, no prior knowledge assumed

Sequences in a batch have different lengths, so short ones are **padded** to the longest length. A **mask** marks which steps are real (1) and which are padding (0). On a padding step the states must pass through unchanged. Day 61 did this for the RNN's one state; the LSTM has two, and **both** must be carried.

REAL-WORLD SCENARIO

A group of hikers walks a trail together; one stops at the 3 km mark while the rest go on to 8 km. Their pedometer and their water bottle must both freeze at the 3 km readings. If only the pedometer stops and the water keeps being drunk, the record says “3 km” but the bottle tells a different story.

MATHEMATICS

$$h_t = m_t \cdot h_{new} + (1 - m_t) \cdot h_{t-1} \qquad c_t = m_t \cdot c_{new} + (1 - m_t) \cdot c_{t-1} \qquad m_t = 1 \text{ real, } 0 \text{ padding}$$

SPECIFICATION

Setup	6 hidden units, 2 inputs. A 3-step sequence alone, then the same sequence padded to 8 steps in a batch with a full 8-step neighbour.
Versions	Mask on h and c (correct); no mask (padding treated as real zero inputs); mask on h only (a plausible bug).
Measured	How far each version's final h and final c are from the unpadded run.

Why this test

Forgetting the mask on one of two states is an easy mistake to make, and it hides well: if a test only looks at the hidden state, it passes. The third version exists to show exactly that.

IMPLEMENTATION

```
m_t = mask[:, t:t + 1]
h = m_t * h_new + (1.0 - m_t) * h
c = m_t * c_new + (1.0 - m_t) * c
# the 'mask on h only' version, written out in demo_masking()
for t in range(8):
    h_new, c, _ = lstm_step_forward(batch[:, t], h, c, W, b)
    h = mask[:, t:t + 1] * h_new + (1 - mask[:, t:t + 1]) * h
```

ACTUAL OUTPUT

```
version                    max |h_final - alone|  max |c_final - alone|
mask on h and c                         5.55e-17               1.11e-16
no mask                                 1.76e-01               4.07e-01
mask on h only                          5.55e-17               3.67e-01
Actual output from this lesson's run.
```

What it means

With both states masked, the padded run matches the unpadded one to rounding (5.55e-17 in h, 1.11e-16 in c). With no mask, both are wrong (1.76e-01 and 4.07e-01).

The h-only version is the instructive one. Its final hidden state is exactly right (5.55e-17) while its cell state is off by 3.67e-01. Any test that compared only h would call it correct. The damage is real all the same: tomorrow's backward pass reads c, and so does any later step that continues from these states.

WHY CHOOSE THIS?

Carrying both states through padding makes a sentence's result independent of what else is in its batch, which is required for correct training and evaluation.

WHY NOT THIS? (TRADEOFFS)

Every step still costs a full cell computation even when the mask throws it away. Sorting sentences by length into batches reduces the wasted work.

3. A faster forward pass? Precomputing the input part

DEFINITION — start here, no prior knowledge assumed

Each gate's raw score is $\mathbf{x}_t \mathbf{W}_x + \mathbf{h}_{t-1} \mathbf{W}_h + \mathbf{b}$. The first part does not depend on earlier steps, so it can be computed for **all** steps in one big matrix product before the loop. Only $\mathbf{h}\,\mathbf{W}_h$ has to stay inside the loop. Libraries such as cuDNN do this.

A **BLAS library** (here OpenBLAS) is the code NumPy calls for matrix products. It can split one product across several CPU **threads**, which helps for big products and can hurt for small ones.

REAL-WORLD SCENARIO

Doing the washing-up for a week: you can wash each plate as it is used (the loop), or wash the plates that don't depend on anything in one big batch at the end of the day. The batch saves effort only if setting up the sink costs more than the plates; for three plates it makes no difference.

Hiring 16 helpers to carry one box is slower than carrying it yourself: they spend the time deciding who holds which corner. That is what happened to the mid-sized products below.

MATHEMATICS

$$A = X_{\text{all steps}}\, W_x + b \qquad \text{one product, shape (examples} \times \text{steps, 4H)}$$

$$\text{inside the loop:}\quad a_t = A_t + h_{t-1}\, W_h \qquad \text{instead of}\quad [x_t, h_{t-1}]\, W + b$$

SPECIFICATION

Correctness	16 sequences, 40 steps, 10 inputs, 12 hidden units: the two versions' h and c compared.
Timing	Five layer sizes from (1 input, 16 hidden) to (300 inputs, 128 hidden); each version timed 7 times and the fastest kept.
Threads	Every size timed twice: with the BLAS library's default number of threads, and limited to one thread with `threadpoolctl`.
Follow-up	The single per-step matrix product of each version timed on its own at D = H = 128, under both thread settings.

Why this test

The first run of this section, with default settings only, showed the precomputed version about 8× faster at the larger sizes (the final run below shows 11×). That looked too good: at D = H = 128, precomputing removes half of the per-step product, which cannot make the whole loop 8× faster. So each per-step product was timed on its own, and with threads limited to one. The second timing is the answer to the question this section actually asks.

IMPLEMENTATION

```
def lstm_forward_fast(X, h0, c0, W, b):
    m, T, D = X.shape
    H = h0.shape[1]
    Wx, Wh = W[:D], W[D:]
    A = (X.reshape(m * T, D) @ Wx + b).reshape(m, T, 4 * H)
    A = A.transpose(1, 0, 2).copy()
    Hs, Cs = np.zeros((m, T, H)), np.zeros((m, T, H))
    h, c = h0, c0
    for t in range(T):
        a = A[t] + h @ Wh
        f, i = sigmoid(a[:, :H]), sigmoid(a[:, H:2 * H])
        g, o = np.tanh(a[:, 2 * H:3 * H]), sigmoid(a[:, 3 * H:])
        c = f * c + i * g
        h = o * np.tanh(c)
        Hs[:, t], Cs[:, t] = h, c
    return Hs, Cs
# inside demo_fast_forward(): the same two calls with BLAS on one thread
with threadpool_limits(limits=1, user_api="blas"):
    t_loop1 = best_time(lstm_forward, X, h0, c0, W, b)
    t_fast1 = best_time(lstm_forward_fast, X, h0, c0, W, b)
```

The subtle part: `A.transpose(1, 0, 2).copy()` puts time first and makes a real copy, so `A[t]` is one contiguous block of memory instead of a scattered view.

ACTUAL OUTPUT

```
same result? max difference in h 1.9e-16, in c 3.3e-16
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
matrix library: openblas, using 16 threads by default
                      ---- default threads ----   ------ one thread ------
   D    H batch steps     loop  precomp speed-up     loop  precomp speed-up
   1   16   200   200   0.0372   0.0392     0.95   0.0363   0.0400     0.91
  32   32    32   100   0.0071   0.0073     0.98   0.0072   0.0073     0.99
  64   64    32   100   0.0216   0.0159     1.35   0.0147   0.0164     0.90
 128  128    32   100   1.3340   0.1164    11.46   0.1414   0.1029     1.37
 300  128    64    50   1.7405   0.2542     6.85   0.0762   0.0575     1.33
Times in seconds, best of 7 runs. Actual output from this lesson's run; timings vary by machine.
```

ACTUAL OUTPUT

```
one matrix product per step, at D = H = 128, microseconds per call:
product                              default threads  one thread
[x, h] @ W  (32x256 @ 256x512)              35266.3       421.0
h @ Wh      (32x128 @ 128x512)               1805.1       318.2
Actual output from this lesson's run.
```

What it means

The two versions give the same numbers (largest difference 1.9e-16 in h).

With one thread, precomputing helps a little, and only for large layers. At the three smaller sizes the speed-up is 0.91, 0.99 and 0.90, i.e. none; at the two largest it is 1.37 and 1.33. That matches what the arithmetic predicts: a modest saving, worth having for big layers.

The large default-thread speed-up is mostly not the trick. With the library's default 16 threads, one step's product in the loop version took 35266.3 microseconds; on a single thread the same product took 421.0. The threaded library is dramatically slower on this mid-sized product, and the precomputed version simply avoids most of it. The default-thread column (11.46 at D = H = 128) measures that, not the precomputation.

A practical consequence, not tested here: the scripts in this series do many mid-sized matrix products, so they may run faster on this machine with the environment variable `OPENBLAS_NUM_THREADS=1`. Timings depend on the machine; measure before relying on it.

WHY CHOOSE THIS?

Precomputing the input projection is free to do, never changes the result, and saves real time when the input is wide (for example word embeddings of a few hundred numbers).

WHY NOT THIS? (TRADEOFFS)

For small layers it saves nothing, and it needs memory for all T steps of projections at once. More generally: a speed-up that looks too large deserves a look at what was actually timed.

4. What the forward pass produces over 200 steps

DEFINITION — start here, no prior knowledge assumed

The cell state is updated by **c = f · c + i · g**. If the forget gate f is close to 1, almost nothing is removed at each step while i · g keeps adding a little: the cell state behaves like a running total. The hidden state is **o · tanh(c)**. When |c| is large, tanh(c) is stuck near ±1: it is **saturated**.

REAL-WORLD SCENARIO

A bathtub with the plug almost fully in (forget gate near 1) and the tap dripping (input gate). Leave it long enough and it fills up, even though no single drip was large. A gauge that only reads “empty” to “full” (tanh) stops telling you anything once the tub is full: every further change reads “full”.

MATHEMATICS

$$\text{with } f \approx 1: \quad c_T \approx c_0 + \Sigma_t \, i_t \, g_t \qquad \text{a sum of T small writes, which can grow without limit}$$

$$\text{slope of tanh at c:} \quad 1 - \tanh(c)^2 \qquad |\tanh(c)| > 0.99 \Rightarrow \text{slope} < 0.02$$

SPECIFICATION

Setup	16 units, 200 sequences of 200 N(0,1) inputs, random weights (seed 3, as on Day 63). Only the forget gate's bias changes: 0, 1, 2, 4.
Measured	At steps 1, 10, 50, 200: mean and largest |c|, the fraction of units with |tanh(c)| > 0.99, and the mean |h|.
Nothing trained	These are untrained weights; a trained network chooses its own gate values.

Why this test

Day 63 showed what a high forget bias buys: a nudge to the memory survives for 100 steps. It also showed the total sensitivity growing at bias 4 (8.78e+01 after 50 steps). The forward pass can show part of the reason directly, without any gradients: what happens to the cell state itself.

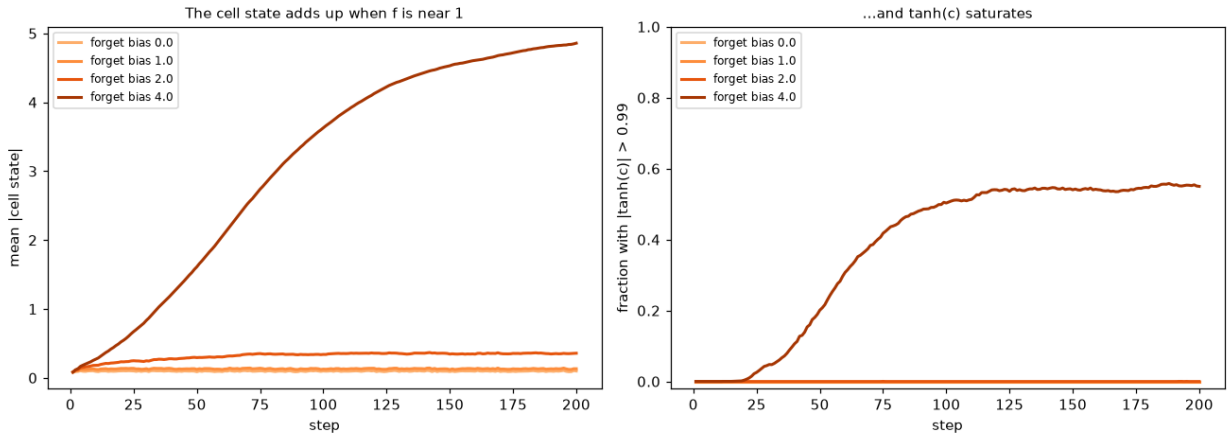
IMPLEMENTATION

```
for fb in [0.0, 1.0, 2.0, 4.0]:
    W, b = init_lstm(1, H, seed=3, forget_bias=fb)
    Hs, Cs, _ = lstm_forward(X, np.zeros((m, H)), np.zeros((m, H)), W, b)
    saturated = np.abs(np.tanh(Cs)) > 0.99
```

```
    curves[fb] = (np.abs(Cs).mean(axis=(0, 2)), saturated.mean(axis=(0, 2)))
```

ACTUAL OUTPUT

```
 f-bias  step  mean |c|   max |c|  saturated  mean |h|
    0.0     1     0.084      0.59       0.0%     0.042
    0.0    10     0.095      0.83       0.0%     0.047
    0.0    50     0.089      0.88       0.0%     0.044
    0.0   200     0.100      0.92       0.0%     0.049
    1.0     1     0.084      0.59       0.0%     0.042
    1.0    10     0.124      1.00       0.0%     0.060
    1.0    50     0.124      1.19       0.0%     0.060
    1.0   200     0.133      1.21       0.0%     0.064
    2.0     1     0.084      0.59       0.0%     0.042
    2.0    10     0.182      1.54       0.0%     0.086
    2.0    50     0.294      1.85       0.0%     0.133
    2.0   200     0.358      1.78       0.0%     0.155
    4.0     1     0.084      0.59       0.0%     0.042
    4.0    10     0.271      2.27       0.0%     0.121
    4.0    50     1.612     10.12      20.1%     0.354
    4.0   200     4.858     26.03      55.0%     0.410
Actual output from this lesson's run.
```



Left: mean size of the cell state at each step. Right: fraction of units whose tanh(c) is saturated.

What it means

Forget bias 0, 1 and 2: the cell state settles. Its mean size stays below 0.358 at step 200 and no unit saturates.

Forget bias 4: it keeps adding up. The mean |c| goes from 0.271 at step 10 to 1.612 at step 50 and 4.858 at step 200, with one value reaching 26.03. By step 200, 55.0% of the units have a saturated tanh(c).

Why that matters. A unit with saturated tanh(c) shows only ±o: further changes in its memory no longer reach h, and the slope that carries gradient through tanh(c) is below 0.02. This is consistent with Day 63's growing sensitivity at bias 4 (a running total magnifies every change) and shows the other side of it. A high forget bias is not free: memory that is never cleared fills up.

WHY CHOOSE THIS?

Watching the cell state during a forward pass costs nothing and shows directly whether memory is settling, accumulating or saturating, before any training is attempted.

WHY NOT THIS? (TRADEOFFS)

These are untrained weights. A trained LSTM learns to close its forget gate when it needs to clear memory, so this is a picture of a starting point, not of a finished model.

5. What the cache costs

DEFINITION — start here, no prior knowledge assumed

To compute gradients, the backward pass needs the value of every gate at every step. Storing them is the **cache**. Its size grows with batch size × sequence length × hidden size, and for an LSTM there are several arrays of that size instead of the RNN's one.

REAL-WORLD SCENARIO

Recording a football match for later analysis. One camera on the ball (the RNN's hidden states) is cheap. Following the ball plus four specific players (the cell state and four gates) needs five times the storage, and for a long match that is what fills the disk.

MATHEMATICS

LSTM, per example per step: D + 6H numbers (x, h, c, f, i, g, o) *RNN: D + H numbers (x, h)*

SPECIFICATION

Setup	Batch 32, 100 steps, 32 inputs, 128 hidden units, float64 (8 bytes per number).
Measured	The real size of each cached array, read with `.nbytes`, for the LSTM and for an RNN of the same size.

Why this test

Memory, not arithmetic, is often what limits how long a sequence or how big a batch can be trained. Before writing the backward pass it is worth knowing what it will have to hold.

IMPLEMENTATION

```
lstm_parts = {k: cache[k].nbytes for k in ["X", "H", "C", "F", "I", "G", "O"]}
rnn_parts = {"X": X.nbytes, "H": Hr.nbytes}
print(f"\n{'array':<7}{'LSTM (MB)':>11}{'RNN (MB)':>10}")
for k in ["X", "H", "C", "F", "I", "G", "O"]:
    print(f"{k:<7}{lstm_parts[k] / 1e6:>11.2f}{(rnn_parts[k] / 1e6 if k in rnn_parts else
        0):>10.2f}")
total_l, total_r = sum(lstm_parts.values()), sum(rnn_parts.values())
```

ACTUAL OUTPUT

```
array    LSTM (MB)  RNN (MB)
X             0.82      0.82
H             3.28      3.28
C             3.28      0.00
F             3.28      0.00
I             3.28      0.00
G             3.28      0.00
O             3.28      0.00
total        20.48      4.10    ratio 5.0
per example per step: LSTM 800 numbers (D + 6H), RNN 160 numbers (D + H)
Actual output from this lesson's run.
```

What it means

The LSTM's cache is 20.48 MB against the RNN's 4.10 MB: ratio 5.0 at these sizes, 800 numbers per example per step against 160. Doubling the sequence length doubles both. The tanh(c) values are not stored; the backward pass will recompute them from C.

WHY CHOOSE THIS?

Caching every gate makes the backward pass a simple walk through stored numbers, with no repeated forward computation.

WHY NOT THIS? (TRADEOFFS)

Five times the memory of an RNN here. For very long sequences, practical systems trade time for memory: they store only some steps and recompute the rest during the backward pass.

Interview preparation — technical questions

Q: What does an LSTM forward pass need to store for backpropagation?

A: The inputs, the starting states, every hidden state, every cell state and the four gate values at every step. Here that came to D + 6H numbers per example per step, five times an RNN's cache at these sizes.

Q: How do you handle padded sequences in an LSTM?

A: With a mask: on padding steps both h and c are carried forward unchanged. Masking only h leaves c wrong while h looks correct; here that bug left c off by 0.37 with h exact.

Q: What can be computed outside the time loop?

A: The input projection x_t W_x + b for every step, in one matrix product. Only h_{t-1} W_h depends on the previous step. It gave the same numbers and was about 1.3x faster for the large layers on one thread.

Q: A change makes your code 8x faster. What do you check?

A: What was actually timed. Here most of the large speed-up (8x in a first run, 11x in the final one) came from the multi-threaded BLAS library being very slow on one mid-sized product (35 ms with 16 threads, 0.4 ms with one). Time each part separately and under controlled thread settings before crediting the change.

Q: Why can the LSTM loop not be parallelised over time?

A: Each step needs the previous step's h and c. Only the input projections, which do not depend on earlier steps, can be done for all steps at once.

Interview preparation — theoretical questions

Q: What happens to the cell state when the forget gate stays near 1?

A: It behaves like a running sum of the writes i * g, so its size can keep growing. With forget bias 4 the mean |c| reached 4.9 after 200 steps and 55% of units had saturated tanh(c).

Q: Why is a saturated tanh(c) a problem?

A: tanh is nearly flat there, so changes in c barely change h, and the gradient through tanh(c), 1 - tanh^2, is below 0.02. The unit stops responding even though its memory is still changing.

Q: Why does an LSTM use more memory than an RNN during training?

A: It has a second state and four gates per step, all needed by the backward pass, against one hidden state for the RNN.

Q: Is a high forget bias always good?

A: No. It lets memory survive (Day 63), but memory that is never cleared accumulates and saturates. A moderate bias such as 1 or 2 is the usual starting point, and training adjusts it.

Key takeaway and what's next

> **KEY TAKEAWAY**
> The LSTM forward pass is Day 63's step inside a loop that carries two states and stores every gate. It is checked exactly against hand-chained steps, handles padding by carrying both h and c, and costs about five times an RNN's memory. Precomputing the input projections is correct and modestly faster for big layers. Run for 200 steps with the forget gate near 1, the cell state keeps adding up until tanh(c) saturates: the price side of Day 63's result.

- Loop: (h, c) = step(x_t, h, c) for every t; store H, C and the gates F, I, G, O.
- Padding: carry both h and c. An h-only mask passes an h-only test while c is wrong.
- x_t W_x can be precomputed for all steps; only h W_h must stay in the loop.
- Time the parts, and control BLAS threads, before believing a large speed-up.
- With f near 1 the cell state is a running sum; it can grow and saturate tanh(c).
- Cache cost per example per step: D + 6H numbers for the LSTM, D + H for the RNN.

Suggested watch order, if starting from zero

- 1. CS231n Winter 2016: Lecture 10: Recurrent Neural Networks, Image Captioning, LSTM (Andrej Karpathy) — RNNs and LSTMs from the people who taught them at Stanford, including the gradient-flow argument.
- 2. Implementing LSTM in numpy from scratch! (Ather Cheema) — an LSTM forward pass written in NumPy, to compare with today's code.

Day 65 preview

- **LSTM backward pass from scratch, with a gradient check.** Walking backward through today's cache, with two gradients flowing back in time (one for h, one for c) and the forget gate appearing in the c-gradient exactly as the direct route of Day 63. After that, the LSTM can be trained.
- Today's exercise script, `day64_exercise.py`, is the forward loop that Day 65's exercise will differentiate.